

JavaScript Array Methods

| Introduction

An **Array** is a data structure used to store multiple values in a single variable.

```
const fruits = ["Apple", "Banana", "Mango"];
```

JavaScript provides many built-in array methods for adding, removing, searching, transforming, filtering, and processing array elements.

This guide covers **17 essential JavaScript Array methods and properties** with definitions, how they work, and two practical examples for each.

| 1. length

| Definition

The `length` property returns the total number of elements in an array.

| How It Works

It counts the elements starting from index `0`. It can also be used to determine the last valid index.

| Example 1: Count Array Elements

```
const students = ["Rahim", "Karim", "Hasan", "Nabil"];

console.log(students.length);
```

Output:

```
4
```

| Example 2: Check Whether an Array Is Empty

```
const cart = [];

if (cart.length === 0) {
  console.log("Your cart is empty");
}
```

Output:

```
Your cart is empty
```

| 2. `push()`

| Definition

`push()` adds one or more elements to the **end** of an array.

| How It Works

The original array is modified, and the method returns the new length of the array.

| Example 1: Add a Product

```
const cart = ["Laptop", "Mouse"];

cart.push("Keyboard");

console.log(cart);
```

Output:

```
["Laptop", "Mouse", "Keyboard"]
```

| Example 2: Add Multiple Students

```
const students = ["Rahim", "Karim"];

students.push("Hasan", "Nabil");

console.log(students);
```

Output:

```
["Rahim", "Karim", "Hasan", "Nabil"]
```

3. `pop()`

Definition

`pop()` removes the **last element** from an array.

How It Works

The original array is modified, and the removed element is returned.

Example 1: Remove the Last Cart Item

```
const cart = ["Laptop", "Mouse", "Keyboard"];

const removedItem = cart.pop();

console.log(removedItem);
console.log(cart);
```

Output:

```
Keyboard
["Laptop", "Mouse"]
```

Example 2: Remove the Last Task

```
const tasks = ["Study", "Practice", "Exercise"];

tasks.pop();

console.log(tasks);
```

Output:

```
["Study", "Practice"]
```

| 4. `shift()`

| Definition

`shift()` removes the **first element** from an array.

| How It Works

All remaining elements move one position toward the beginning. The removed element is returned.

| Example 1: Process a Queue

```
const queue = ["Customer 1", "Customer 2", "Customer 3"];

const servedCustomer = queue.shift();

console.log(servedCustomer);
console.log(queue);
```

Output:

```
Customer 1
["Customer 2", "Customer 3"]
```

| Example 2: Remove the First Notification

```
const notifications = ["New Message", "Friend Request", "New Comment"];

notifications.shift();

console.log(notifications);
```

Output:

```
["Friend Request", "New Comment"]
```

5. `unshift()`

Definition

`unshift()` adds one or more elements to the **beginning** of an array.

How It Works

Existing elements move one position forward. The method returns the new length of the array.

Example 1: Add a New Priority Task

```
const tasks = ["Study", "Exercise"];

tasks.unshift("Complete Assignment");

console.log(tasks);
```

Output:

```
["Complete Assignment", "Study", "Exercise"]
```

Example 2: Add a New Message

```
const messages = ["Hello", "How are you?"];

messages.unshift("New Message");

console.log(messages);
```

Output:

```
["New Message", "Hello", "How are you?"]
```

| 6. `includes()`

| Definition

`includes()` checks whether an array contains a specific value.

| How It Works

It returns `true` if the value exists and `false` if it does not.

| Example 1: Check Product Availability

```
const products = ["Laptop", "Mouse", "Keyboard"];

console.log(products.includes("Mouse"));
```

Output:

```
true
```

| Example 2: Check User Permission

```
const permissions = ["read", "write", "delete"];

if (permissions.includes("delete")) {
  console.log("User can delete data");
}
```

Output:

```
User can delete data
```

| 7. `indexOf()`

| Definition

`indexOf()` returns the index of the **first occurrence** of a specified value.

| How It Works

If the value is not found, it returns `-1`.

Example 1: Find a Product

```
const products = ["Laptop", "Mouse", "Keyboard"];

const index = products.indexOf("Mouse");

console.log(index);
```

Output:

```
1
```

Example 2: Check Whether a User Exists

```
const users = ["Rahim", "Karim", "Hasan"];

const index = users.indexOf("Karim");

if (index !== -1) {
  console.log("User found");
}
```

Output:

```
User found
```

8. `slice()`

Definition

`slice()` returns a **shallow copy** of a portion of an array without changing the original array.

Syntax

```
array.slice(start, end);
```

The `end` index is not included.

Example 1: Get a Page of Products

```
const products = [
  "Laptop",
  "Mouse",
  "Keyboard",
  "Monitor",
  "Headphone"
];

const firstPage = products.slice(0, 3);

console.log(firstPage);
```

Output:

```
["Laptop", "Mouse", "Keyboard"]
```

Example 2: Get Selected Students

```
const students = ["Rahim", "Karim", "Hasan", "Nabil"];

const selectedStudents = students.slice(1, 3);

console.log(selectedStudents);
```

Output:

```
["Karim", "Hasan"]
```

9. `splice()`

Definition

`splice()` adds, removes, or replaces elements in an array.

How It Works

Unlike `slice()`, `splice()` **modifies the original array**.

Syntax

```
array.splice(start, deleteCount, item1, item2, ...);
```

Example 1: Remove an Item From a Cart

```
const cart = ["Laptop", "Mouse", "Keyboard"];

cart.splice(1, 1);

console.log(cart);
```

Output:

```
["Laptop", "Keyboard"]
```

Example 2: Replace a Product

```
const products = ["Laptop", "Mouse", "Keyboard"];

products.splice(1, 1, "Monitor");

console.log(products);
```

Output:

```
["Laptop", "Monitor", "Keyboard"]
```

10. `forEach()`

Definition

`forEach()` executes a function once for every element in an array.

| How It Works

It is mainly used when you want to **perform an action** for every element. It does not create a new array.

| Example 1: Display Student Names

```
const students = ["Rahim", "Karim", "Hasan"];

students.forEach(student => {
  console.log(student);
});
```

Output:

```
Rahim
Karim
Hasan
```

| Example 2: Calculate Total Cart Price

```
const prices = [500, 1000, 1500];

let total = 0;

prices.forEach(price => {
  total += price;
});

console.log(total);
```

Output:

```
3000
```

| 11. `map()`

| Definition

`map()` creates a **new array** by applying a function to every element of the original array.

| How It Works

The original array remains unchanged, while the returned array contains the transformed values.

| Example 1: Apply a Discount

```
const prices = [1000, 2000, 3000];

const discountedPrices = prices.map(price => price * 0.9);

console.log(discountedPrices);
```

Output:

```
[900, 1800, 2700]
```

| Example 2: Get User Names

```
const users = [
  { name: "Rahim", age: 22 },
  { name: "Karim", age: 25 },
  { name: "Hasan", age: 21 }
];

const names = users.map(user => user.name);

console.log(names);
```

Output:

```
["Rahim", "Karim", "Hasan"]
```

| 12. filter()

| Definition

`filter()` creates a new array containing only the elements that satisfy a condition.

| How It Works

If the callback returns `true`, the element is included in the new array. If it returns `false`, it is excluded.

Example 1: Find Adults

```
const ages = [15, 18, 21, 25, 16, 30];

const adults = ages.filter(age => age >= 18);

console.log(adults);
```

Output:

```
[18, 21, 25, 30]
```

Example 2: Find Available Products

```
const products = [
  { name: "Laptop", stock: 5 },
  { name: "Mouse", stock: 0 },
  { name: "Keyboard", stock: 10 }
];

const availableProducts = products.filter(product => product.stock > 0);

console.log(availableProducts);
```

Output:

```
[
  { name: "Laptop", stock: 5 },
  { name: "Keyboard", stock: 10 }
]
```

13. find()

Definition

`find()` returns the **first element** that satisfies a condition.

How It Works

Once the first matching element is found, the search stops. If nothing matches, it returns `undefined`.

Example 1: Find a User

```
const users = [
  { id: 1, name: "Rahim" },
  { id: 2, name: "Karim" },
  { id: 3, name: "Hasan" }
];

const user = users.find(user => user.id === 2);

console.log(user);
```

Output:

```
{ id: 2, name: "Karim" }
```

Example 2: Find a Product by Name

```
const products = [
  { name: "Laptop", price: 70000 },
  { name: "Mouse", price: 1000 },
  { name: "Keyboard", price: 2500 }
];

const product = products.find(product => product.name === "Mouse");

console.log(product);
```

Output:

```
{ name: "Mouse", price: 1000 }
```

14. `some()`

Definition

`some()` checks whether **at least one** element satisfies a condition.

How It Works

It returns `true` as soon as it finds one matching element. If no element matches, it returns `false`.

Example 1: Check for an Out-of-Stock Product

```
const products = [
  { name: "Laptop", stock: 5 },
  { name: "Mouse", stock: 0 },
  { name: "Keyboard", stock: 10 }
];

const hasOutOfStock = products.some(product => product.stock === 0);

console.log(hasOutOfStock);
```

Output:

```
true
```

Example 2: Check for a Failing Student

```
const marks = [75, 82, 45, 90];

const hasFailed = marks.some(mark => mark < 50);

console.log(hasFailed);
```

Output:

```
true
```

15. every()

Definition

`every()` checks whether **all elements** satisfy a condition.

How It Works

It returns `true` only when every element passes the condition.

Example 1: Check Whether All Students Passed

```
const marks = [65, 72, 80, 90];

const allPassed = marks.every(mark => mark >= 40);

console.log(allPassed);
```

Output:

```
true
```

Example 2: Check Whether All Products Are Available

```
const products = [
  { name: "Laptop", stock: 5 },
  { name: "Mouse", stock: 3 },
  { name: "Keyboard", stock: 10 }
];

const allAvailable = products.every(product => product.stock > 0);

console.log(allAvailable);
```

Output:

```
true
```

16. `reduce()`

Definition

`reduce()` processes all array elements and reduces them to **one final value**.

How It Works

It is commonly used for calculating totals, counting items, grouping data, or building a single result from an array.

| Syntax

```
array.reduce((accumulator, currentValue) => {  
  return accumulator;  
}, initialValue);
```

| Example 1: Calculate Total Price

```
const prices = [500, 1000, 1500];  
  
const total = prices.reduce((sum, price) => {  
  return sum + price;  
}, 0);  
  
console.log(total);
```

Output:

```
3000
```

| Example 2: Calculate Total Cart Quantity

```
const cart = [  
  { product: "Laptop", quantity: 1 },  
  { product: "Mouse", quantity: 2 },  
  { product: "Keyboard", quantity: 1 }  
];  
  
const totalItems = cart.reduce((total, item) => {  
  return total + item.quantity;  
}, 0);  
  
console.log(totalItems);
```

Output:

4

17. `sort()`

Definition

`sort()` sorts the elements of an array and **modifies the original array**.

How It Works

By default, JavaScript sorts values as strings. For numbers, use a comparison function.

Example 1: Sort Prices

```
const prices = [3000, 500, 1500, 1000];

prices.sort((a, b) => a - b);

console.log(prices);
```

Output:

```
[500, 1000, 1500, 3000]
```

Example 2: Sort Students by Marks

```
const students = [
  { name: "Rahim", marks: 75 },
  { name: "Karim", marks: 90 },
  { name: "Hasan", marks: 60 }
];

students.sort((a, b) => b.marks - a.marks);

console.log(students);
```

Output:

```
[
  { name: "Karim", marks: 90 },
  { name: "Rahim", marks: 75 },
  { name: "Hasan", marks: 60 }
]
```

Quick Comparison

Method	Main Purpose	Changes Original Array?	Returns
<code>length</code>	Count elements	No	Number
<code>push()</code>	Add to end	Yes	New length
<code>pop()</code>	Remove from end	Yes	Removed element
<code>shift()</code>	Remove from beginning	Yes	Removed element
<code>unshift()</code>	Add to beginning	Yes	New length
<code>includes()</code>	Check value	No	Boolean
<code>indexOf()</code>	Find index	No	Number
<code>slice()</code>	Copy part of array	No	New array
<code>splice()</code>	Add/remove/replace	Yes	Removed elements
<code>forEach()</code>	Run code for each element	No*	<code>undefined</code>
<code>map()</code>	Transform elements	No	New array
<code>filter()</code>	Select matching elements	No	New array
<code>find()</code>	Find first matching element	No	Element / <code>undefined</code>
<code>some()</code>	Check if any match	No	Boolean
<code>every()</code>	Check if all match	No	Boolean
<code>reduce()</code>	Create one final result	No	Any value
<code>sort()</code>	Sort elements	Yes	Sorted array

* `forEach()` itself does not change the array, although the callback can modify objects or other mutable data stored inside it.

`map()` vs `filter()` vs `find()` vs `forEach()`

These four methods are especially important for modern JavaScript and React.

`map()`

Use when you need a **new transformed array**.

```
const numbers = [1, 2, 3];

const doubled = numbers.map(num => num * 2);
```

filter()

Use when you need **multiple matching elements**.

```
const numbers = [10, 20, 30, 40];

const result = numbers.filter(num => num >= 30);
```

find()

Use when you need the **first matching element**.

```
const numbers = [10, 20, 30, 40];

const result = numbers.find(num => num >= 30);
```

forEach()

Use when you simply need to **perform an action for every element**.

```
const names = ["Rahim", "Karim", "Hasan"];

names.forEach(name => {
  console.log(name);
});
```

Important Concepts to Remember

Mutating Methods

These methods modify the original array:

```
push()  
pop()  
shift()  
unshift()  
splice()  
sort()
```

| Non-Mutating Methods

These do not modify the original array:

```
includes()  
indexOf()  
slice()  
map()  
filter()  
find()  
some()  
every()  
reduce()
```

| Methods That Return a New Array

```
slice()  
map()  
filter()
```

| Methods That Return a Boolean

```
includes()  
some()  
every()
```

Methods That Return One Element or Value

```
pop()
shift()
find()
reduce()
```

Recommended Learning Order

For practical JavaScript development, learn the methods in this order:

```
length
↓
push() / pop()
↓
shift() / unshift()
↓
includes() / indexOf()
↓
slice() / splice()
↓
forEach()
↓
map()
↓
filter()
↓
find()
↓
some() / every()
↓
reduce()
↓
sort()
```

Key Takeaway

If you are learning JavaScript for **React, Node.js, or full-stack development**, focus especially on:

```
map()  
filter()  
find()  
reduce()  
forEach()  
some()  
every()  
sort()
```

A strong understanding of these methods will make working with API data, objects, React components, tables, dashboards, shopping carts, and backend data much easier.